

UTC503

La Programmation Logique

Le paradigme logique⁽¹⁾

- **Approche déclarative:** « Le quoi plutôt que le comment »
 - **Opposé à l'impératif :** on ne dit pas comment obtenir le résultat
 - **Description du problème à résoudre par :**
 - La liste des objets concernés
 - Leurs propriétés
 - Les relations qu'ils vérifient
- **La résolution du problème** est effectué par un « moteur d'inférence » qui évalue toutes les possibilités et fournit les solutions du problème qui sont acceptables (conforme à sa description)

Historique de la programmation logique ⁽¹⁾

■ La Démonstration automatique :

- **Vers 1930** : Herbrand (mathématicien et logicien français) pose les conditions de validité d'une démonstration automatique.
- **En 1965** : Robinson proposait une méthode exprimant les énoncés logique sous forme clausale et proposant une règle pour la résolution. (mise en pratique par PROLOG)
- **En 1958** : John Mac Carthy (inventeur du Lisp) propose d'utiliser la logique comme langage déclaratif. (cogniticien rédacteur des règle + moteur d'inférence)

Historique de la programmation logique (2)

■ L'avènement de Prolog (PROgrammation en LOGique) :

- **1970** : Alain Colmeurauer (informaticien français) reprend l'idée de la logique comme langage de programmation. Création de Q-systèmes, utilisé pour la traduction automatique de l'anglais vers le français à base de règles.
- **En 1972** : Alain Colmeurauer et P. Roussel créent Prolog comme « Outil permettant de décrire un monde en français et de traiter ensuite des question sur ce monde ». Une application a été l'interrogation de bases de données en langage naturel.
- **En 1977**: D. Warren développa à Édimbourg un compilateur Prolog, qui apporta à Prolog la performance qui lui manquait. Le Prolog d'Édimbourg devint ainsi un standard.

Historique de la programmation logique (3)

■ Cas d'utilisation de la programmation logique en Prolog :

- **Traitement des langages naturels par ordinateur** : traduction, dialogue
- **Intelligence artificielle**: tentative de créer un ordinateur de 5^{ème} génération (avec les Japonais)
- **Les systèmes experts** : programme pour résoudre des problèmes très difficiles aussi bien ou mieux que des experts humains. (Détermination d'itinéraires, Tracé de route, Diagnostic médical, Evaluation des risques, etc.) Grande mode au milieu des années 80.
- **Résolution de logigrammes et autres casses têtes**:
« Max a un chat. Eric n'est pas en pavillon. Luc habite un studio mais le cheval n'y est pas »
Chacun habite une maison différente et possède un animal distinct.
Qui habite le château et qui a le poisson ?

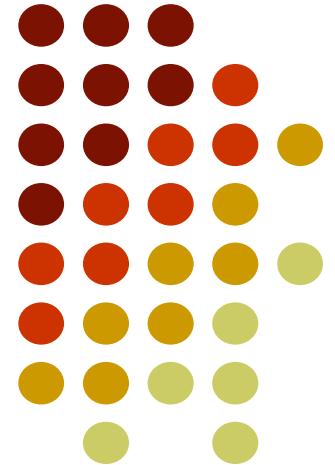


Prolog

Partie 1 : Notions de base

Jean-Michel Adam

Université Grenoble Alpes
L2 - MIAHS
Grenoble - France





Plan du cours

- 1 – Introduction à la programmation logique
- 2 – Le langage Prolog
- 3 – Les listes
- 4 – Les graphes et les arbres en Prolog
- 5 – Grammaires et automates d'états finis en Prolog



Partie 1

-

Introduction

Prolog



- Prolog est un langage de programmation à part.
- Le nom Prolog vient de **Program**mation **Log**ique.
- Il est utilisé principalement en **Intelligence Artificielle**.
- Ce qui est original, c'est qu'en Prolog, il suffit
 - de **décrire** ce que l'on sait sur le domaine étudié, (en Intelligence Artificielle, on appelle cela une base de connaissances),
 - puis on pose une **question** à propos de ce domaineet Prolog va nous répondre, sans que l'on ait à lui dire comment construire sa réponse !

Historique



- 1965 : John Robinson décrit les principes théoriques de la programmation logique via la règle de résolution.
- 1972 : création de Prolog par A. Colmerauer et Ph. Roussel à Marseille (Luminy), pour le traitement des langues naturelles
- 1977: premier compilateur par D.H. Warren à Edimbourg
- 1980 : reconnaissance de Prolog comme langage de développement en Intelligence Artificielle
- Depuis, plusieurs versions ont été développées avec un standard ISO, plusieurs extensions possibles.

Les principaux paradigmes en programmation



- Programmation impérative (Java L1)
- Programmation fonctionnelle (Scheme L1)
- Programmation orientée objets (Java L2)
- Programmation déclarative (Prolog L2)



La programmation impérative

- Démarche algorithmique qui décrit la façon de traiter les données pour atteindre un résultat par une série d'actions (instructions).
- L'ordre d'exécution des instructions est impératif : déterminé à l'avance.
- Le déroulement du programme est parfaitement déterministe.
- Importance des structures de contrôle
- Exemples : C, Basic, Pascal, Fortran, Java vu en L1, ...

La programmation fonctionnelle



- La programmation consiste à faire de la composition de fonctions.
- Proche de la programmation impérative ; cependant ses fondements (λ -calcul) ainsi que l'absence de branchements et d'affectation (dans sa forme théorique) en fait un mode de programmation à part.
- Dans la programmation fonctionnelle, on distingue deux grandes familles de langages :
 - les langages fortement typés, ex : ML, Caml, Haskell
 - les langages non typés, ex : Lisp, Scheme

La programmation orientée objet



- Décomposition d'un objet en attributs et méthodes.
- Notions de classe et d'interface, de sous-classe, d'héritage, de polymorphisme
- Exemples : Smalltalk, C++, Java, C#, ActionScript, VisualBasic, Ada, Python.



Programmation déclarative

- La programmation **logique** qui est dite **déclarative**.
- La **programmation déclarative**
 - Le programmeur ne s'occupe pas de la manière d'obtenir le résultat; par contre, le programmeur doit faire la description du problème à résoudre en décrivant les objets concernés, leurs propriétés et les relations qu'ils vérifient.
 - Cette description constitue la **base de connaissance**
 - Ensuite, le mécanisme de résolution intégré au langage, général et universel, parcourt de façon non déterministe toutes les possibilités du problème et calcule les solutions.
 - En Prolog, la résolution s'appuie sur une déduction logique de toutes les conséquences de la base de connaissance via **l'unification**.

Information complémentaires sur Prolog



- La syntaxe et la sémantique du langage Prolog est l'ensemble des règles qui définissent comment un programme Prolog est écrit et interprété.
- Les règles sont énoncées dans la norme ISO/IEC 13211, bien qu'il existe des différences dans les implémentations de Prolog, voir:
http://en.wikipedia.org/wiki/Comparison_of_Prolog_implementations
- Dans ce cours nous utiliserons la syntaxe dite Edinburgh.



Prolog utilisé en TP

- SWI-Prolog
 - Fonctionne sous Linux, Windows et MacOS.
 - Disponible gratuitement (licence BSD) au département informatique de l'Université de Psychologie d'Amsterdam
<http://www.swi-prolog.org/download/stable>
- Autres Prolog gratuits
 - Il y a également Visual Prolog Personal Edition, gratuit pour un usage privé
 - GNU-Prolog (par l'INRIA).



Bibliographie

- **Cours en ligne**

- <http://pcaboche.developpez.com/article/prolog/presentation/>
- <http://www.learnprolognow.org/> (in English)

- **Pour aller plus loin:**

- **Foundations of Logic Programming**, J. W. Lloyd, Springer-Verlag New York, Inc. New York, NY, USA ©1984 ISBN:0-387-13299-6
- **Prolog**, F. Giannesini, H. Kanoui, R. Pasero et M. Van Caneghem, InterÉditions, 1985
- **Programming in Prolog**, W.F. Clocksin, C.S. Mellish, Springer-Verlag Telos; 4th edition (September 1994)
- **The art of Prolog** , 2nd Edition, Advanced Programming Techniques, L. Sterling, E. Shapiro, the MIT Press, 1994, ISBN-13: 978-0-262-19338-2
- **Logic Programming and Prolog (2nd ed)**, Ulf Nilsson and Jan Maluszynski, 2000

Question

- Que veut dire l'acronyme Prolog?



Question



- Quel est le paradigme de programmation lié à Prolog?
 1. Orienté-objet
 2. Fonctionnel
 3. Déclaratif
 4. Impératif

Question

- Que doit faire le programmeur Prolog?



Question



- Comment va répondre un programme Prolog?



Partie 2

-

Bases de connaissance

Construction d'un programme

Prolog



- **Base de connaissance** : description du problème à résoudre.
- Une base de connaissance est composée de **Clauses**:
 - Les **Faits**
 - Les **Règles**
- L'utilisateur pose ensuite une **Question**
- Le programme donne sa **Réponse**

Base de connaissance 1

Collection de Faits



```
mange.  
herbivore(chevre).  
mange(loup, chevre).
```

```
?- mange.  
true.  
?- herbivore(chevre).  
true.  
?- herbivore(loup).  
false.  
?- carnivore(loup)  
ERROR: toplevel: Undefined procedure carnivore/1
```



Les Faits

- Les faits sont des données élémentaires que l'on considère vraies.
- Ce sont des formules atomiques constituées d'un **prédicat** suivi ou pas d'une liste ordonnée d'**arguments** mis entre parenthèses qui sont les objets auxquels s'appliquent le prédicat.
- Un programme Prolog est au moins constitué d'un ou plusieurs faits car c'est à partir d'eux que Prolog va pouvoir rechercher des preuves pour répondre aux requêtes de l'utilisateur
- Ce sont en quelque sorte les hypothèses de travail.



Les Faits

- Généralement, on place toutes les déclarations de faits au début du programme même si ce n'est pas obligatoire.
- Un fait se termine toujours par un point « . »

Exemples :

- Henri IV est le père de Louis XIII se traduit par :
pere(henri4,louis13). ou par : **pere('Henri4','Louis13').**
- Marie de Médicis est la mère de Henri IV se traduit par :
mere(marie-de-medicis,henri4).
ou **mere('Marie de Medicis', 'Henri4').**
- $0! = 1$ se traduit par : **factorielle(0,1).**



Les Faits

Forme : **nomprédictat**(arg₁,arg₂, ..., arg_n).

Les parenthèses et arguments sont optionnels

Exemples:

- univers.
- astre(terre).
- étoile(soleil).
- satellite(terre, soleil).
- satellite(lune, terre).
- capitale('France', 'Paris').

Base de connaissance 2

Introduction des Règles



```
mange(loup, chevre).  
cruel(loup) :- mange(loup, chevre).  
carnivore(loup) :- mange(loup, chevre), animal(chevre).
```

```
?- cruel(loup).  
true.  
?- carnivore(poireau).  
false.  
?- carnivore(loup)  
ERROR: carnivore/1: Undefined procedure: animal/1
```

Pas d'individu poireau => Pas d'évaluation du prédicat carnivore => false

Individu loup => Evaluation du prédicat carnivore => prédicat non défini: animal



Règles Prolog

- Enoncent la dépendance d'un prédicat par rapport à d'autres prédicats
- Concernent des catégories d'objets / faits
 - fait : `enColere(fido).`
 - règle : *Fido aboie si Fido est en colère* s'écrit :
`aboie(fido) :- enColere(fido).`
- Le « **si** » s'écrit « `:-` » en Prolog et correspond à l'implication $\Rightarrow$
$$\text{enColere(fido)} \Rightarrow \text{aboie(fido)}.$$

s'écrit en Prolog : `aboie(fido) :- enColere(fido).`



Règles Prolog

- Un programme Prolog contient presque toujours des règles, cependant ce n'est pas une obligation.
- Si les faits sont les hypothèses de travail, les règles sont des relations qui permettent à partir de ces hypothèses d'établir de nouveaux faits par déduction (si on a démontré $F1$ et $F1 \Rightarrow F2$ alors on a démontré $F2$).
 - Cette règle est appelée en logique le Modus Ponens



Règles Prolog

- Il peut y avoir plusieurs conditions derrière le « :- », séparées par des virgules ou des points-virgules
 - La virgule correspond à un ET logique (conjonction)
 - Le point-virgule correspond à un OU logique (disjonction)
 - Exemple :

La relation telle que si on est le père du père ou de la mère de quelqu'un alors on est son grand-père se traduit par :

```
grandpere(xavier,yves) :- pere(xavier,joe), pere(joe,yves).
```

```
grandpere(xavier,yves) :- pere(xavier,joe), mere(joe,yves).
```

ou encore par :

```
grandpere(xavier,yves) :- pere(xavier,joe), (pere(joe,yves) ; mere(joe,yves)).
```


Base de connaissance 3

Introduction des Variables



```
mange(loup, chevre).  
carnivore(X) :- mange(X, Y), animal(Y).  
cruel(X) :- mange(X, _).
```

```
?- cruel(loup)  
true  
?- carnivore(loup)  
ERROR: carnivore/1: Undefined procedure: animal/1  
?- carnivore(poireau)  
false
```



Constantes vs Variables

- Constantes: commencent par une minuscule ou entre apostrophes ou entiers ou flottants
- Variables : commencent par une majuscule ou par `_`
 - Servent à dénoter une catégorie d'objets
 - Les variables utilisées dans un fait ou une règle sont universellement quantifiées par Prolog, avec le quantificateur $\forall$ (« quel que soit » ou « pour tout »).
 - Les variables utilisées dans une requête sont existentiellement quantifiées par Prolog, avec le quantificateur $\exists$ (« existe-t-il »).



Variables

- Sont utilisées dans les règles afin de les généraliser :
 - *grandpere(xavier,yves) :- pere(xavier,joe), pere(joe,yves).*
 - *grandpere(xavier,yves) :- pere(xavier,joe), mere(joe,yves).*

Généralisé par :

- *grandpere(X,Y) :- pere(X,Z), (pere(Z,Y) ; mere(Z,Y))*
- Peuvent aussi être utilisées dans les faits
Superman est plus fort que tout le monde se traduit par :
plusfort(superman,X).



Variable anonyme

- Notée '_' (tiret bas)
- Représente un objet dont on ne souhaite pas connaître la valeur, c'est une variable muette.
- ?- invite(X,_).
Prolog ne donnera que les valeurs pour X
- Attention, une variable commençant par '_' et de longueur ≥ 2 n'est pas muette par défaut en SWI-Prolog. Activation par :
set_prolog_flag(toplevel_print_anon, false).

Exercice



- Exprimer en Prolog les propositions suivantes, identifier les objets, les faits, les règles
 - *la chèvre est un animal herbivore*
 - *le loup est un animal cruel*
 - *toute chose cruelle est carnivore*
 - *un animal carnivore mange de la viande et un animal herbivore mange de l'herbe*
 - *un animal carnivore mange des animaux herbivores*
 - *les carnivores et les herbivores boivent de l'eau*
 - *un animal consomme ce qu'il boit ou ce qu'il mange*
 - Question : y a-t-il un animal cruel et que consomme-t-il ?



Solution possible

animal(chevre).

herbivore(chevre).

animal(loup).

cruel(loup).

carnivore(X) :- cruel(X).

mange(X,viande):- animal(X), carnivore(X).

mange(X,herbe):- animal(X), herbivore(X).

mange(X,Y):- animal(X), animal(Y), carnivore(X),herbivore(Y).

boit(X,eau):- animal(X), (carnivore(X); herbivore(X)).

consomme(X,Y) :- mange(X,Y); boit(X,Y).

?- animal(X), cruel(X), consomme(X,Y).

la chèvre est un animal herbivore

le loup est un animal cruel

toute chose cruelle est carnivore

un animal carnivore mange de la viande et un animal herbivore mange de l'herbe

un animal carnivore mange des animaux herbivores

les carnivores et les herbivores boivent de l'eau

un animal consomme ce qu'il boit ou ce qu'il mange

Question : y a-t-il un animal cruel et que consomme-t-il ?



Partie 3

-

Mécanisme de Résolution



Résolution d'une requête

- Comment Prolog répond à une requête?
- 3 mécanismes principaux
 - Règle de généralisation
 - Règle d'instanciation
 - Règle de résolution



Règle de généralisation

- Sert à traiter les variables dans les requêtes

Exemple: ?- cruel(X).

- Loi de généralisation: $p(a)$ donc $\exists X$ tq: $p(X)$

Exemple : $\text{cruel}(\text{loup})$ donc $\exists X$ tq: $\text{cruel}(X)$

$X = \text{loup}$

- Prolog cherche donc un fait qui est une instance de la requête. Il va essayer de remplacer les variables par des constantes.



Règle d'instanciation

- Sert à traiter les variables dans les faits = faits universels

Exemple: `plusMalin(arseneLupin,Y)`.

- Loi d'instanciation: $\forall X p(X)$ donc $p(a)$
- Exemple: `plusMalin(arseneLupin,policier)`
- Prolog remplace les variables des faits universels par toutes les constantes de la base de connaissance



Règle de résolution

- Sert à la déduction de nouveaux faits à partir d'anciens faits et des règles

- Exemple: `cruel(X) :- tue(X,Y).`
 `tue(lion, gazelle).`

Permet de déduire `cruel(lion).`

- Modus Ponens Général:
 - `F :- F1, F2,..., Fn`
 - Pour démontrer `F`, il faut démontrer `F1`, et `F2`,..., et `Fn`.
- Règle de résolution permet d'appliquer le modus ponens



Représenter les Faits et Règles

- Forme générale : $F :- F1, F2, \dots, Fn.$
- C'est une clause de Horn*
- F est la tête de la clause
- $F1, F2, \dots, Fn$ est appelée la queue de la clause
- Un fait = clause dont la queue est vide
 - aime(X, bach). « Pour tout X, X aime Bach. »
- Une règle = clause dont la queue est non vide.
 - a_un_salaire(X) :- employeur(_,X).

* Leur nom vient du logicien Alfred Horn qui a remarqué leur importance en 1951 42



Exercice

- Démontrer en utilisant les règles de généralisation, d'instanciation et le modus ponens que la requête a au moins une solution avec la base de connaissance suivante .

(pas besoin d'appliquer en détail la règle de résolution)

aimeSkier(paul).

aEquipement(zoe).

neige(X).

damePiste(chamrousse).

ski(X) :- stationOuverte(Y), aEquipement(X).

stationOuverte(X) :- neige(X), damePiste(X).

?-ski(X).



Partie 4

-

Syntaxe Prolog

Atomes



- Chaîne de caractères commençant par une **lettre minuscule** et contenant des lettres, chiffres et le caractère de soulignement ()
- Chaîne de caractères entre **guillemets simple** : 'Vincent', 'Léo Ferré'
 - Permet l'utilisation d'espaces
- Symboles spéciaux :
:= et \= et ; et :-
- Certains de ces atomes ont une signification prédéfinie (opérateurs).

Nombres



- Prolog définit des entiers et des réels. Nous nous concentrerons sur les entiers ici.
- Entiers : -4, 567, 0, -34
 - 0xA2C2, 0xffff (hexadécimal)
 - 0o777, 0o11 (octal)
 - 0b1111, 0b10 (binaire)
- Réels : 2.183720 (point décimal)
1.32e-12 (flottant)

Variables



- Chaîne de caractère démarrant par une **lettre majuscule** ou le **caractère de soulignement**
- Peut contenir des lettres, chiffres ou le caractère de soulignement
- Expl : X, Y, _var
- _ est appelée la **variable anonyme**

Termes complexes



- **Terme simple** : atome, nombre ou variable
- Un **terme complexe** est construit via un **foncteur** suivi d'une séquence d'**arguments**
- **Foncteur** (nom de prédicat) : **atome**
- **Argument** : **terme simple ou complexe**
- Les arguments sont placés après le foncteur, entre parenthèses et séparés par des virgules
- L'**arité** d'un foncteur est son nombre n d'arguments. Indiquée en suffixant par $/n$
- Exemple : donne/2 diffère de donne/3

Opérateurs de comparaison (1)

- On a les opérateurs de comparaison habituels

Opérateurs	Description
$X = := Y$	égalité de deux variables
$X \neq Y$	non égalité de deux variables
$X < Y$	strictement plus petit
$X > Y$	strictement plus grand
$X \leq Y$	plus petit ou égal
$X \geq Y$	plus grand ou égal

- Noter que $X = Y$ et $X =:= Y$ n'ont pas le même comportement :

```
?- +(4,3) = 7.
```

```
false.
```

```
?- +(4,3) =:= 7.
```

```
true.
```

Opérateurs de comparaison (2)

- $T1 == T2$ réussit si $T1$ est identique à $T2$;
- $T1 \backslash == T2$ réussit si $T1$ n'est pas identique à $T2$;
- $T1 = T2$ unifie $T1$ avec $T2$;
- $T1 \backslash = T2$ réussit si $T1$ n'est pas unifiable à $T2$.

```
?- f(X) == f(x) .
```

```
false.
```

```
?- f(X) = f(x) .
```

```
X = x
```

```
?- a(G,7) \== a(6,7) .
```

```
true.
```

Exercice



- Parmi les chaînes de caractère suivantes, lesquelles sont des atomes, lesquelles sont des variables et lesquelles ne sont ni l'un ni l'autre ?
 1. `VINCENT`
 2. `Football`
 3. `variable23`
 4. `Variable2000`
 5. `petit_pain`
 6. `'petit pain'`
 7. `petit pain`
 8. `'Jules'`
 9. `_Jules`
 10. `'_Jules'`



Partie 5

-

Unification



Unification : Idée de base

- Exemple :
Base de connaissance : `elfe(galadriel)`.
Question : `?- elfe(X)`.
- Comment fait Prolog pour dire que galadriel est une valeur possible pour X ?
- **Unification** : procédé par lequel on essaie de rendre deux formules identiques en donnant des valeurs aux variables qu'elles contiennent.
- Unification réussie : substitution possible



Unification dans les Clauses

- L'unification réussie dans la tête se propage aux clauses

frere(X,Y) :- homme(X), enfant(X,Z), enfant(Y,Z), X\=Y.

frere(patrick, Qui).

Unification avec frere(X,Y)

X=patrick et Y=Qui

homme(patrick)
enfant(patrick,Z)
enfant(Qui,Z)
patrick\=Qui



Définition Précise

- Si t_1 et t_2 sont des **constantes**, t_1 et t_2 unifient ssi ils sont le même atome ou le même nombre.
- Si t_1 est une **variable** et t_2 quelconque, t_1 et t_2 unifient ssi t_1 est instanciable en t_2 .
- Si t_1 et t_2 sont des **termes complexes**, t_1 et t_2 unifient ssi :
 - Ils ont le même foncteur et même arité
 - Tous leurs arguments correspondants unifient
 - Les instanciations de variables sont compatibles

Conséquence



- L'unification peut réussir ou échouer.
 - $e(X)$ et $e(2)$ peuvent être unifiés.
 - $e(X,X)$ et $e(2,3)$ ne peuvent être unifiés
- Le résultat n'est pas forcément unique : ensemble de solutions
- Prédicat d'unification : « = ». Teste si l'unification a réussi.
 - Ecriture 1 : $=(a,b)$.
 - Ecriture 2 : $a=b$.
- Unification optimiste : $\text{pere}(X)=X$ unifie
- Unification pessimiste, n'unifie pas :
`unify_with_occurs_check(pere(X),X).`



Exercice

- Dire si les expressions suivantes unifient et comment :
 - $?- \text{sam} = \text{sam}.$
 - $?- \text{sam} = \text{lea}.$
 - $?- \text{'sam'} = \text{sam}.$
 - $?- \text{'1'} = 1 .$
 - $?- X = Y.$
 - $?- X = \text{sam}, X = \text{lea}.$
 - $?- k(s(g), Y) = k(X, t(k)).$
 - $?- k(s(g), t(k)) = k(X, t(Y)).$
 - $?- \text{aime}(X, X) = \text{aime}(\text{sam}, \text{lea}).$



Démonstration Prolog (1)

- Pour répondre à une question, Prolog cherche à unifier la question, soit avec un fait, soit avec la tête d'une règle.
- Si l'unification avec la tête d'une règle réussit : substitution des variables de la queue par les valeurs correspondantes dans la tête.
- L'algorithme est appelé SLDNF résolution (Selective Linear Definite Negation as Failure)

Résultat Final



- Prolog continue à unifier jusqu'à vérification de la requête initiale
- Si l'unification échoue : Prolog répond faux (false)
- Si l'unification réussit : Prolog répond soit vrai (true), soit donne la valeur des variables (Exemple : $X=lea$) s'il y en avait dans la requête.
- On demande à Prolog de chercher d'autres valeurs en appuyant sur « ; » ou « Espace »
- A la fin de la démonstration, Prolog a construit une liste de couples valeur-variable

Parcours



- **Parcours de la base du haut vers le bas**
- **Parcours des clauses de gauche à droite**
- Quand il y a plusieurs manières d'unifier, on a un **point de choix**. Cas possibles :
 - Différentes règles définissant un même prédicat
 - Disjonction de prédicats dans une queue
 - Instanciation d'une variable (plusieurs possibilités)



Points de choix

- Prolog effectue des essais consécutifs dans l'ordre de déclaration des prédicats
- Cette séquence d'essais peut être représentée par un arbre: l'arbre de recherche
- Les nœuds de l'arbre sont les points de choix



Arbre de recherche (1)

- Pour résoudre une question, Prolog construit l'arbre de recherche de la question :
 - Racine de l'arbre : la question initiale
 - Nœuds : points de choix (formules à démontrer)
 - Passage d'un nœud vers son successeur en effectuant une unification



Arbre de recherche (2)

- Les nœuds sont démontrés de gauche à droite, dans l'ordre de déclaration dans la règle
- Nœuds d'échec : aucune règle ne permet de démontrer la première formule du nœud
- Nœuds de succès : ne contient plus aucune formule à démontrer, tout a été démontré et les éléments de solution sont trouvés en remontant vers la racine de l'arbre

Arbre de recherche (3)



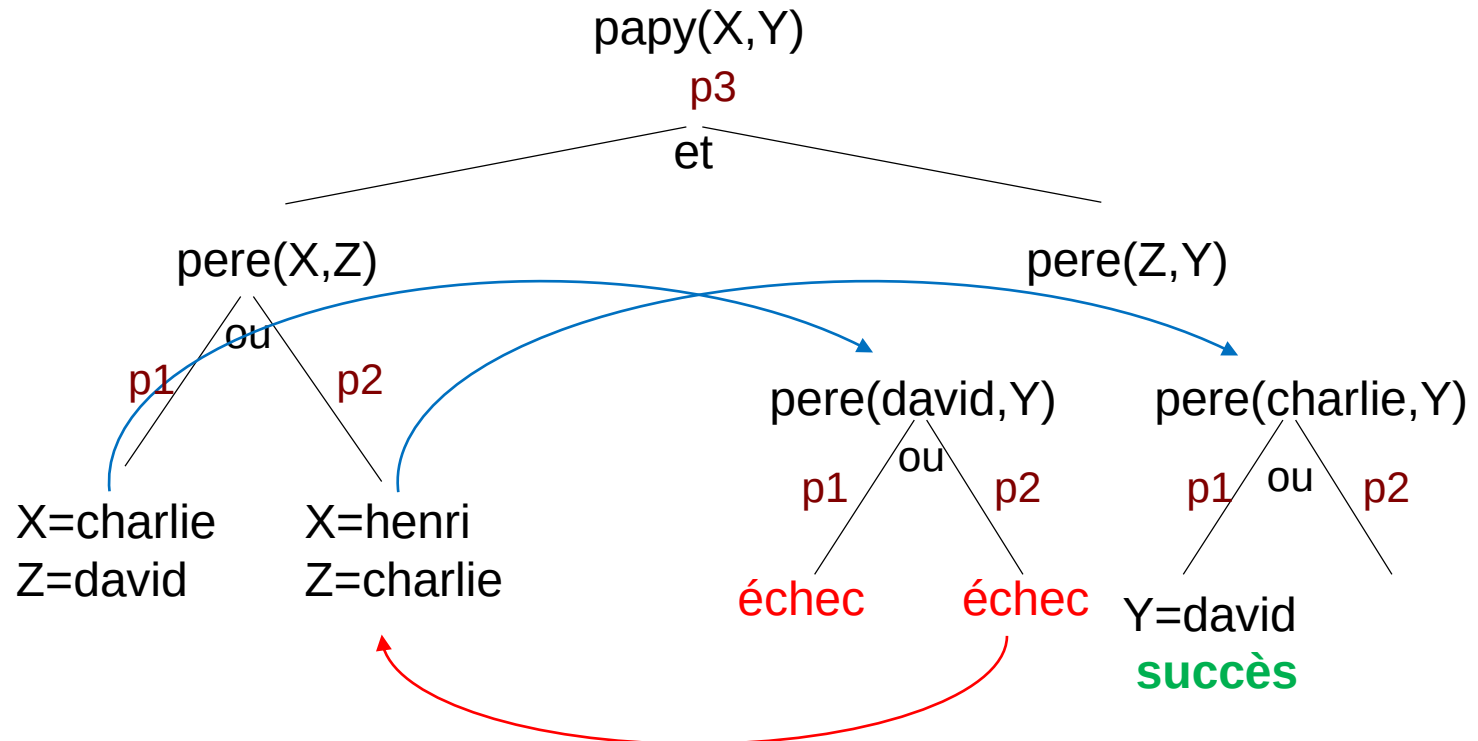
Exemple de programme:

pere(charlie, david). (p1)

pere(henri, charlie). (p2)

papy(X,Y) :- pere(X,Z), pere(Z,Y). (p3)

Appel du programme : papy(X,Y).





Stratégie de Prolog

- Parcours en profondeur d'abord
- nœud de succès : c'est une solution, Prolog l'affiche et peut chercher d'autres solutions
- nœud d'échec : remontée dans l'arbre jusqu'à un point de choix possédant des branches non explorées
- Puis parcours de gauche à droite dans l'ordre des conditions



Backtracking

- Remontée dans l'arbre jusqu'au point de choix : **backtrack**. Si un tel nœud de choix n'existe pas, la démonstration est terminée, il n'y a pas d'autre solution.
- **Bactrack** : toutes les unifications faites depuis le dernier point de choix sont défaites
- Possibilité de branche infinie et donc de recherche sans terminaison... Attention à :
 - L'ordre des littéraux dans la queue de clause
 - L'ordre des clauses

Exemple de démonstration



`planete(X) :- astre(X), etoile(Y), satellite(X,Y).`

faits:

`astre(lune).`

`astre(terre).`

`astre(soleil).`

`astre(venus).`

`satellite(venus, soleil).`

`satellite(lune, terre).`

`satellite(terre, soleil).`

`etoile(vega).`

`etoile(soleil).`

?- `planete(X).`

Exemple de démonstration



planete(X) :- astre(lune), etoile(Y), satellite(lune,Y).

faits:

astre(lune).

astre(terre).

astre(soleil).

astre(venus).

satellite(venus, soleil).

satellite(lune, terre).

satellite(terre, soleil).

étoile(vega).

étoile(soleil).

Exemple de démonstration



planete(X) :- astre(lune), etoile(vega), satellite(lune, vega).

faits:

échec !

astre(lune).

astre(terre).

astre(soleil).

astre(venus).

satellite(venus, soleil).

satellite(lune, terre).

satellite(terre, soleil).

étoile(vega).

étoile(soleil).

Exemple de démonstration



planete(X) :- astre(lune), etoile(soleil), satellite(lune, soleil).

faits: backtrack échec !

astre(lune).

astre(terre).

astre(soleil).

astre(venus).

satellite(venus, soleil).

satellite(lune, terre).

satellite(terre, soleil).

étoile(vega).

étoile(soleil).

Exemple de démonstration



planete(X) :- astre(terre), etoile(Y), satellite(terre, Y).

faits: backtrack

astre(lune).

astre(terre).

astre(soleil).

astre(venus).

satellite(venus, soleil).

satellite(lune, terre).

satellite(terre, soleil).

étoile(vega).

étoile(soleil).

Exemple de démonstration



planete(X) :- astre(terre), etoile(vega), satellite(terre, vega).

faits:

échec !

astre(lune).

astre(terre).

astre(soleil).

astre(venus).

satellite(venus, soleil).

satellite(lune, terre).

satellite(terre, soleil).

étoile(vega).

étoile(soleil).

Exemple de démonstration



planete(X) :- astre(terre), etoile(soleil), satellite(terre, soleil).

faits:

backtrack

true !

astre(lune).

astre(terre).

astre(soleil).

astre(venus).

satellite(venus, soleil).

satellite(lune, terre).

satellite(terre, soleil).

étoile(vega).

étoile(soleil).

X = terre,

Y = soleil

Exemple de démonstration



planete(X) :- astre(soleil), etoile(Y), satellite(soleil, Y).

faits: backtrack échec !

astre(lune).

astre(terre).

astre(soleil).

astre(venus).

satellite(venus, soleil).

satellite(lune, terre).

satellite(terre, soleil).

étoile(vega).

étoile(soleil).

Exemple de démonstration



planete(X) :- astre(venus), etoile(Y), satellite(venus, Y).

faits: backtrack

astre(lune).

astre(terre).

astre(soleil).

astre(venus).

satellite(venus, soleil).

satellite(lune, terre).

satellite(terre, soleil).

étoile(vega).

étoile(soleil).

Exemple de démonstration



planete(X) :- astre(venus), etoile(vega), satellite(venus, vega).

faits: échec !

astre(lune).

astre(terre).

astre(soleil).

astre(venus).

satellite(venus, soleil).

satellite(lune, terre).

satellite(terre, soleil).

étoile(vega).

étoile(soleil).

Exemple de démonstration



planete(X) :- astre(venus), etoile(soleil), satellite(venus,soleil).

faits:

true !

astre(lune).

X = venus,

astre(terre).

Y = soleil

astre(soleil).

astre(venus).

satellite(venus, soleil).

satellite(lune, terre).

satellite(terre, soleil).

étoile(vega).

étoile(soleil).



Exercice

- Soit la base de connaissance et la requête suivante.
Construire l'arbre de recherche Prolog.

```
rectangle(a).  
rectangle(b).  
losange(b).  
carre(X) :- rectangle(X),losange(X).
```

```
?- carre(X).
```


UTC503

La Programmation Logique

Démonstrations et Exercices

Utilisation de l'interpréteur Prolog ⁽¹⁾

Télécharger et installer SWI-Prolog : <http://www.gecif.net/articles/linux/swi-prolog.exe>

1 - Ecriture de la base de connaissance : avec un éditeur de texte dans un fichier avec extension .pl

Exemple :

% Le chien est un animal et le chat également

`animal(chien).`

`animal(chat).`

% paul pierre et jean sont des prénoms

`prenom(paul).`

`prenom(pierre).`

`prenom(jean).`

Enregistrer le fichier sous le nom **premierEssai.pl**

Utilisation de l'interpréteur Prolog (2)

2 – Lancer SWI PROLOG et charger le fichier par l'option « Consult... »

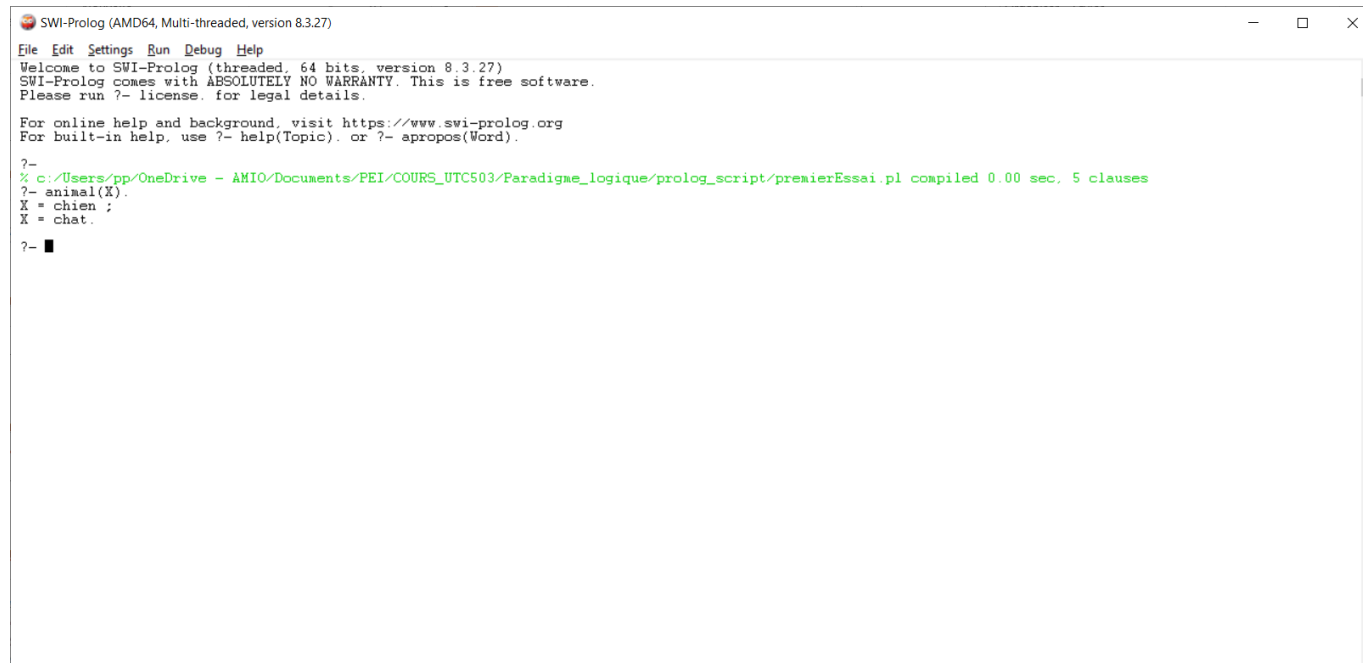


Utilisation de l'interpréteur Prolog (3)

3 – Requête : « Qui est un animal ? » :

animal(X). [*+ Entrée*] retourne le premier résultat.

Pour obtenir les autres résultats appuyer sur la touche « ; »



```
SWI-Prolog (AMD64, Multi-threaded, version 8.3.27)
File Edit Settings Run Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 8.3.27)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?-
% c:/Users/pp/OneDrive - AMIO/Documents/PEI/COURS_UTC503/Paradigme_logique/prolog_script/premierEssai.pl compiled 0.00 sec, 5 clauses
?- animal(X).
X = chien ;
X = chat.
?-
```

La mère de Jean

- **Les données du problème sont :** *Alice et Luc sont mariés. Luc est le père de Jean.*
- **La problématique à résoudre est :** *Qui est la mère de Jean ?*

Les faits sont :

- alice est l'épouse de luc
- luc est le père de jean

Les règles sont:

- l'épouse du père d'un enfant est la mère de cet enfant

En prolog:

```
% Les faits  
epouse(alice,luc).  
pere(luc,jean).
```

```
% règles : l'épouse du père est la mère  
mere(M,E) :- pere(P,E), epouse(M,P).
```

Manger des fruits ⁽¹⁾

■ Les données du problème sont :

- Adam aime les pommes
- Clara aime les carottes
- Olivier aime les oranges
- Les pommes sont des fruits
- Les oranges sont des fruits
- Les carottes sont des légumes
- Ceux qui aiment les fruits sont en bonne santé

■ La problématique à résoudre est : *Qui est en bonne santé ?*

En prolog ...

Manger des fruits (2)

■ En Prolog :

```
aime(adam,pomme).  
aime(clara,carotte).  
aime(olivier,orange).  
fruit(pomme).  
fruit(orange).  
legumes(carotte).  
bonne_sante(X):-aime(X,Y),fruit(Y).
```

■ La Requête :

```
bonne_sante(X).
```

Quand on écrit n'importe quoi on obtient...

- Si l'on décrit mal le problème la solution sera erronée

% Les chats sont mortels

homme(X) :- mortel(X).

chat(X) :- mortel(X).

% Socrate est mortels

mortel(socrate).

% donc...

Un premier logigramme ⁽¹⁾

■ Les données du problème sont :

Max a un chat

Eric n'est pas en pavillon

Luc habite un studio mais le cheval n'y est pas

Chacun habite une maison différente et possède un animal distinct

■ La problématique à résoudre est :

Qui habite le château et qui a le poisson ?

■ Les faits sont :

Il existe 3 maisons différentes : un château, un studio et un pavillon

Il existe 3 animaux différents : un chat, un poisson et un cheval

Un premier logigramme (2)

■ En prolog:

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%  
% Réalisé par Jean-Christophe MICHEL  
% Juillet 2011  
% www.gecif.net  
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%  
  
%-----  
% les faits :  
  
% les 3 maisons :  
maison(chateau).  
maison(studio).  
maison(pavillon).  
  
% les 3 animaux :  
animal(chat).  
animal(poisson).  
animal(cheval).
```

Un premier logigramme ⁽³⁾

% les règles :

% le prédicat relation constitue la relation entre une personne, son animal et sa maison :

relation(max,M,chat):-maison(M).

relation(luc,studio,A):-animal(A),A\==cheval.

relation(eric,M,A):-maison(M),M\==pavillon,animal(A).

% le prédicat different est vraie seulement si ses 3 paramètres sont différents :

different(X,X,_):-!,fail.

different(X,_,X):-!,fail.

different(_,X,X):-!,fail.

different(_,_,_).

% le prédicat "resoudre" indique les 4 inconnues à retrouver :

resoudre(MM,ME,AE,AL) :- relation(max,MM,chat), relation(eric,ME,AE),
different(MM,ME,studio), relation(luc,studio,AL),
different(AE,AL,chat).

Arbre généalogique (1)

% homme/1

homme(jean). % jean est un homme
homme(fabien). homme(jerome). homme(emile).
homme(franck). homme(bruno). homme(marc).

% femme/1

femme(evelyne). % évelyne est une femme
femme(louise). femme(julie). femme(anne). femme(aurelie).
femme(sophie). femme(marie). femme(eve).

% pere/2

pere(emile,jean). % ´emile est le père de jean
pere(jean,fabien). pere(fabien,eve). pere(jean,jerome).
pere(bruno,evelyne). pere(bruno,franck). pere(franck,sophie).
pere(franck,aurelie).

% mere/2

mere(louise,jean). % louise est la mère de jean
mere(julie,evelyne). mere(julie,franck).
mere(evelyne,fabien). mere(evelyne,jerome).
mere(anne,sophie). mere(anne,aurelie).
mere(marie,eve). mere(sophie,marc).

Arbre généalogique (2)

Définir les prédicats suivants :

(a) parent(X,Y)	% X est un des parents de Y
(b) fils(X,Y)	% X est le fils de Y
(c) fille(X,Y)	% X est la fille de Y
(d) femme(X,Y)	% X est la femme de Y
(e) mari(X,Y)	% X est le mari de Y
(f) gdPere(X,Y)	% X est le grand-pere de Y
(g) gdMere(X,Y)	% X est la grand-mere de Y
(h) gdParent(X,Y)	% X est un des grand-parents de Y
(i) aGdPere(X,Y)	% X est l'arriere grand-pere de Y
(j) aGdMere(X,Y)	% X est l'arriere grand-mere de Y
(k) frereSoeur(X,Y)	% X est le frere ou la soeur de Y
(l) frere(X,Y)	% X est le frere de Y
(m) soeur(X,Y)	% X est la soeur de Y
(n) beauFrere(X,Y)	% X est le beau-frere de Y
(o) belleSoeur(X,Y)	% X est la belle-soeur de Y
(p) ancetre(X,Y)	% X est l'ancetre de Y

Listes Prolog ⁽¹⁾

- une liste en Prolog est:
 - soit la constante “liste vide”: le terme `[]`
 - soit un terme à deux arguments:
le premier élément (*head*) et le reste de la liste (*tail*).
Ce terme s'écrit `[head|tail]`
- `head` est un terme quelconque, `tail` doit être une liste.
Les listes ne sont pas homogènes:
`[3|['hello'|[]]]`
est une liste de longueur 2, dont le premier élément est un entier
et le deuxième un string.
- On peut aussi énumérer les éléments d'une liste, séparés par des virgules. La liste précédente peut s'écrire `[3,'hello']`

Opérations sur les listes en Prolog : Appartenance d'un élément

- Appartenance: `member(X,L)` où `X` est un terme et `L` une liste.
- Le but `member(X,L)` réussit si `X` est un élément de `L`.
- `member(b,[a, b, c])` et `member([b, c],[a, [b, c]])` sont vrais.
`member(b,[a, [b,c]])` est faux.
- On peut observer que `X` est membre de `L`, si
 - `X` est le *head* de `L`, ou bien
 - `X` est membre du *tail* de `L`
- En Prolog:
`member( X, [X | _ ]).`
`member( X, [ _ | L] ) :- member( X, L).`

Opérations sur les listes en Prolog : Concaténation

- `append(L1,L2,L3)` est vrai si et seulement si la liste `L3` est la concatenation de `L1` et `L2`.
- Pour définir `append(L1,L2,L3)` on peut considérer deux cas, en fonction de la forme du premier argument `L1`:
 - Si `L1` est vide, alors `L2` et `L3` doivent être deux listes égales.
 - Si `L1` est non vide, alors `L1` est de la forme `[X|L]`. Dans ce cas concaténer `L1` avec `L2` donne une liste `[X|L3]` où `L3` est la concaténation de `L` et `L2`.
- Cela donne en PROLOG:

`append([],L,L).`

`append([X|L],L2,[X|L3]) :- append(L,L2,L3).`

`append([a,b,c],[1,2,3],[a,b,c,1,2,3]).`

`append([a,b,c],[1,2,3],L).`

`append([a,[b,c],[[]]],[a,[],L]).`

Opérations sur les listes en Prolog : Retrait d'un élément

Enlever un élément:

- Enlever l'élément X d'une liste L :
`delete(X,L,L1)`
où $L1$ est la liste L sans l'élément X
- `delete(X, [X|Q], Q) .`
`delete(X, [Y|Q], [Y|Q1]) :- delete(X,Q,Q1) .`
- Si X apparaît plusieurs fois dans la liste, des retours en arrière successifs donnent toutes les possibilités.
- `delete(X,L,L1)` échoue si X n'apparaît pas dans L .

Exemples d'utilisation de `delete`:

```
delete(a, L, [c, d, e]).
```

```
L = [a, c, d, e]
```

Opérations sur les listes en Prolog : Ajout d'un élément

Insérer un élément dans une liste:

- En utilisant `delete` on peut définir `insérer(X,L,L1)` qui insère l'élément `X` dans la liste `L`, de toute les manières possibles. La liste `L1` est le résultat de l'insertion.
`insérer(X,L,L1) :- delete(X,L1,L).`
- On peut aussi redéfinir le prédicat `member` en utilisant `delete`. Un élément `X` est membre d'une liste si on peut l'enlever de cette liste. `membre2(X,L) :- delete(X,L,_).`

Références :

- Cours Prolog de l'IRIF <https://www.irif.fr/>
- Cours de Prolog Jean-Michel Adam Université Grenoble Alpes
- Cours de Ange Tato
- Exemples de problèmes résolus en Prolog <http://www.gecif.net/articles/linux/prolog.html>
- Cours de Prolog avec Turbo Prolog : Eléments fondamentaux